

Different Service Lifetimes in Dependency Injection In .NET, the Dependency Injection (DI) container provides three primary service lifetimes. Understanding these lifetimes is critical for managing memory and ensuring that objects are reused or created as required.

Singleton: A single instance of the service is created and shared throughout the entire lifetime of the application. This is useful for services that maintain state or are expensive to initialize (e.g., database connections or cache systems).

`builder.Services.AddSingleton<INotificationService, EmailNotificationService>();` When to

Use: When your service is stateless or you want the same instance to be used globally

across the application. 2. **Scoped:** A new instance of the service is created per HTTP request (in the context of web applications). Each request gets its own service instance, but the same instance is used throughout the life of that request.

`builder.Services.AddScoped<INotificationService, EmailNotificationService>();` When to

Use: When you need to maintain some form of state across a single request but want

isolation between requests. 3. **Transient:** A new instance of the service is created every time it is requested from the container. This is useful for lightweight, stateless services that do not maintain state between invocations.

`builder.Services.AddTransient<INotificationService, EmailNotificationService>();` When to

Use: When the service is stateless and you want a new instance every time to ensure no residual state is carried over between uses. explain this

ChatGPT said:

Sure! Let's break it down and explain **Dependency Injection (DI) lifetimes** in .NET with simple terms and examples.

What is Dependency Injection (DI)?

Dependency Injection is a design pattern that helps you manage dependencies (like services or classes your code needs) in a **centralized and flexible** way. Instead of manually creating instances everywhere in your code, you register them once, and the **DI container** handles creating and providing them when needed.

.NET's built-in DI container offers three **service lifetimes**:

1. Singleton

◇ Definition:

- Only **one instance** is created **for the entire lifetime of the application**.
- That **same instance is reused everywhere** it is requested.

◇ Code Example:

```
csharp
CopyEdit
builder.Services.AddSingleton<INotificationService,
EmailNotificationService>();
```

◇ When to Use:

- When the service is **stateless** or **needs to share state globally**.
- Example: **Caching services, logging, configuration readers**.

◇ Visual Analogy:

Imagine a single **coffee machine** in an office that everyone shares all day.

2. Scoped

◇ Definition:

- A **new instance is created for each HTTP request** (in web apps).
- Within a single request, the **same instance** is reused.
- Isolated across requests, but consistent within one.

◇ Code Example:

```
csharp
```

```
CopyEdit
builder.Services.AddScoped<INotificationService,
EmailNotificationService>();
```

◇ When to Use:

- When you need to **maintain data/state during one request** (like tracking a user's action).
- Example: **Database context (DbContext)** in Entity Framework.

◇ Visual Analogy:

Think of a **coffee mug per person per day**. One person uses one mug all day, then throws it out.

3. Transient

◇ Definition:

- A **new instance is created every single time** it is requested.
- No sharing, no reuse.

◇ Code Example:

```
csharp
CopyEdit
builder.Services.AddTransient<INotificationService,
EmailNotificationService>();
```

◇ When to Use:

- When the service is **very lightweight** and **stateless**.
- Example: A simple utility that formats strings or sends notifications.

♦ Visual Analogy:

Every time someone wants coffee, a **new cup is made**, and discarded after use.

Summary Table

Lifetime	Instance Per Request	Shared Globally	Best Use Case
Singleton	✗	✓	Caching, Logging, Config Readers
Scoped	✓ (per request)	✗	DB Context, User Session Data
Transient	✓ (always new)	✗	Light, Stateless Helpers/Utilities

Quick Tips

- **Avoid state** in **Singleton** if it's not thread-safe.
- Use **Scoped** for things tied to a user request (like EF Core's DbContext).
- Use **Transient** when you want **fresh** objects each time (no leftovers).

Let me know if you want real-world examples or a diagram to visualize this!

You said:

explain with more example and cases

ChatGPT said:

Absolutely! Let's go deeper with **more examples, use cases, and pros/cons** for each **Dependency Injection lifetime** in .NET — **Singleton**, **Scoped**, and **Transient**.

☑ 1. Singleton

◇ Description:

- One instance is created **once** and reused **everywhere** throughout the app's lifetime.
- Created the **first time it is requested**, and then used for **every other request** or usage.

◇ Code:

```
csharp
CopyEdit
builder.Services.AddSingleton<ILogger, FileLogger>();
```

◇ Real-world Examples:

- **Caching service:**

```
csharp
CopyEdit
builder.Services.AddSingleton<ICacheService, MemoryCacheService>();
```

→ You want a shared cache that persists across all users.

- **Configuration Reader:**

```
csharp
CopyEdit
builder.Services.AddSingleton<IConfigReader, JsonConfigReader>();
```

→ Load config once and reuse it.

- **Logger:**

```
csharp
CopyEdit
```

```
builder.Services.AddSingleton<ILogger, FileLogger>();
```

→ Reuse the same logger instance to log across the app.

☑ When to Use:

- When service is **stateless** or needs to **share state across the app**.
- When the service is **expensive to create** and you want to create it **only once**.

⚠ Be Careful:

- Singleton **lives forever**. If it holds data, that data is shared across all users.
- **Thread-safety is critical**: it may be used by many threads at the same time.

☑ 2. Scoped

◇ Description:

- A new instance is created **once per request** (in ASP.NET Core).
- Within a single HTTP request, all dependencies get the **same instance**.

◇ Code:

```
csharp
CopyEdit
builder.Services.AddScoped<IUserService, UserService>();
```

◇ Real-world Examples:

- **EF Core DbContext**:

```
csharp
CopyEdit
```

```
builder.Services.AddDbContext<MyDbContext>(options =>
    options.UseSqlServer(connectionString));
```

→ EF Core's DbContext should live only for a request. It tracks changes during the request and then is discarded.

- **Services with request-specific logic:**

```
csharp
CopyEdit
builder.Services.AddScoped<ICartService, ShoppingCartService>();
```

→ A cart object for a user's request.

- **Authentication/Session Services:**

```
csharp
CopyEdit
builder.Services.AddScoped<IUserSessionService, UserSessionService>();
```

→ Holds data about the current user during a request.

When to Use:

- When services are **stateful per request**, and you want **isolation** between user requests.
- When service needs to interact with DbContext or needs per-request data.

Be Careful:

- **Don't inject Scoped services into Singletons**, it can cause issues since the Singleton lives longer.

☑ 3. Transient

◇ Description:

- A **new instance is created every single time** the service is requested.
- No caching or reuse.

◇ Code:

```
csharp
CopyEdit
builder.Services.AddTransient<IFormatterService,
HtmlFormatterService>();
```

◇ Real-world Examples:

- **Lightweight utility services:**

```
csharp
CopyEdit
builder.Services.AddTransient<IStringFormatter,
DefaultStringFormatter>();
```

- **Validation or transformation services:**

```
csharp
CopyEdit
builder.Services.AddTransient<IInputSanitizer, DefaultSanitizer>();
```

- **Short-lived logic processing:**

```
csharp
CopyEdit
builder.Services.AddTransient<IEmailComposer, EmailComposer>();
```


☑ When to Use:

- When service is **stateless** and **inexpensive to create**.
- When you don't want any instance reuse.

⚠ Be Careful:

- Too many transient services may increase memory usage if the objects are heavy or deeply nested in dependency trees.

🔄 Lifetime Comparison Table with Use Cases

Lifetime	Created When	Shared?	Lifespan	Good For
Singleton	Once	Yes (app-wide)	Entire application	Config, Logger, Cache, Static Data
Scoped	Per request	Yes (per request)	One HTTP request	DbContext, User Session, Business Logic
Transient	Every time	No	Short-lived	Formatters, Validators, Lightweight Utils

🔍 Mixed Lifetime Dependency Example:

Suppose you have:

```
csharp
CopyEdit
public class ReportService : IReportService
{
    private readonly IDbContext _db;
    private readonly IEmailService _email;

    public ReportService(IDbContext db, IEmailService email)
    {
        _db = db;
    }
}
```

```
        _email = email;
    }
}
```

And you register them as:

```
csharp
CopyEdit
builder.Services.AddScoped<IReportService, ReportService>();
builder.Services.AddScoped<IDbContext, AppDbContext>();
builder.Services.AddTransient<IEmailService, EmailService>();
```

Here:

- ReportService and DbContext will share the same instance per request.
- EmailService will be created **new every time it's used**, even within the same request.

Debug Tip: See the Difference in Action

```
csharp
CopyEdit
public class TestService
{
    public Guid Id { get; } = Guid.NewGuid();
}
```

Register this as Singleton, Scoped, or Transient and inject into a controller:

```
csharp
CopyEdit
public class HomeController : Controller
{
    private readonly TestService _service;
```

```

public HomeController(TestService service)
{
    _service = service;
}

public IActionResult Index()
{
    return Content(_service.Id.ToString());
}
}

```

- Singleton: Same Guid across all requests.
- Scoped: Same Guid in one request, different across others.
- Transient: New Guid every single time.

Let me know if you want a **visual diagram**, **sample project**, or want to see how to test these in Postman or Swagger!

You said:

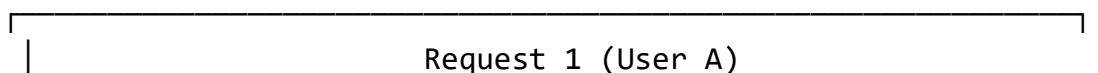
Yes

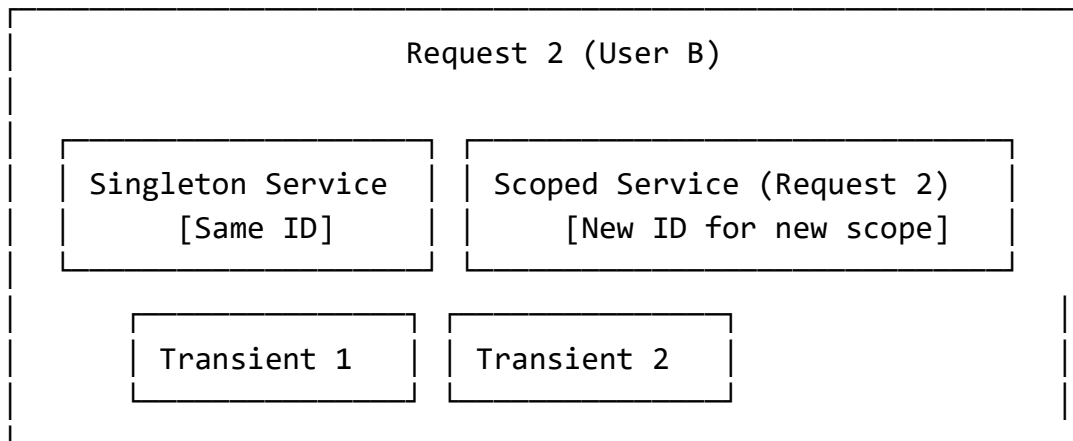
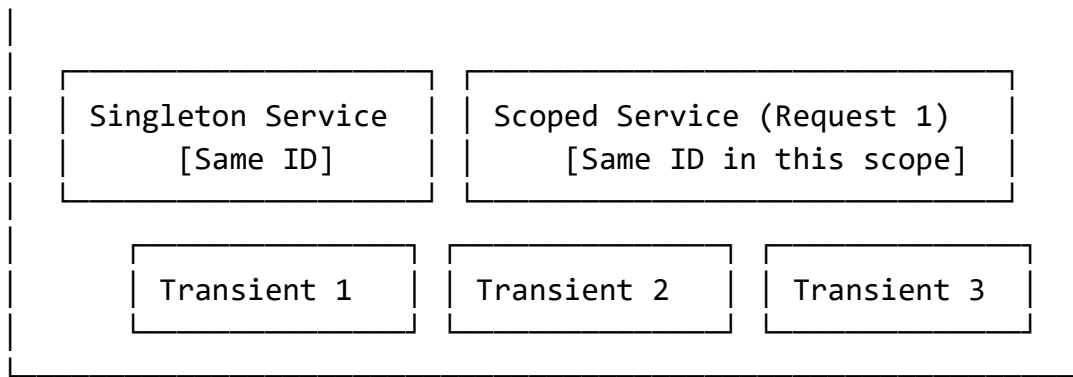
ChatGPT said:

Great! Here's a **visual diagram**, followed by a **small sample .NET project structure** and an explanation on how to **test the different lifetimes** using Swagger or Postman.

Visual Diagram – Service Lifetimes in .NET Dependency Injection

SCSS
CopyEdit





Sample .NET Project to Test Service Lifetimes

1. Create 3 Services to simulate lifetimes

```
csharp
CopyEdit
public interface ILifetimeDemoService
{
    Guid GetGuid();
}

public class LifetimeDemoService : ILifetimeDemoService
{
    private Guid _id;
```

```

    public LifetimeDemoService()
    {
        _id = Guid.NewGuid();
    }

    public Guid GetGuid() => _id;
}

```

2. Register in Program.cs

```

csharp
CopyEdit
builder.Services.AddSingleton<ILifetimeDemoService,
LifetimeDemoService>();    // For Singleton
// builder.Services.AddScoped<ILifetimeDemoService,
LifetimeDemoService>();    // For Scoped
// builder.Services.AddTransient<ILifetimeDemoService,
LifetimeDemoService>();    // For Transient

```

Uncomment the one you want to test.

3. Use in a Controller

```

csharp
CopyEdit
[ApiController]
[Route("[controller]")]
public class LifetimeController : ControllerBase
{
    private readonly ILifetimeDemoService _service1;
    private readonly ILifetimeDemoService _service2;

    public LifetimeController(ILifetimeDemoService service1,
ILifetimeDemoService service2)
    {
        _service1 = service1;
    }
}

```

```

        _service2 = service2;
    }

    [HttpGet]
    public IActionResult Get()
    {
        return Ok(new
        {
            FirstService = _service1.GetGuid(),
            SecondService = _service2.GetGuid()
        });
    }
}

```

How to Test (via Swagger or Postman)

1. **Run the application** (e.g., `dotnet run`).
2. Open browser at: <https://localhost:xxxx/swagger> (or use Postman).
3. Call: GET `/lifetime`

Observe the Guid values returned:

- **Singleton:** Both FirstService and SecondService have the **same Guid** (shared globally).
- **Scoped:** Same Guid **within one request**, but changes on new request.
- **Transient:** Always different even within the same request.

Summary

Lifetime	Created	Same in Request?	Same Across Requests?	Typical Use
Singleton	Once (startup)	✓	✓	Logging, Config, Cache

Scope	Per request	✓	✗	EF DbContext, Session Logic Formatters, Short Utilities
Transi ent	Every injection	✗	✗	

Let me know if you want:

- Full working .NET solution .zip
- SQL-backed scoped example (e.g., DbContext)
- Custom middleware to log lifetime behavior across requests

I'd be happy to help further!

You said:

create pdf of complete session with question asked

ChatGPT said:

Your PDF has been generated successfully. You can download it using the link below:

 [Download Dependency_Injection_Lifetimes_DOTNET.pdf](#)

Let me know if you'd like to include diagrams or code samples in a separate appendix!

You said:

pdf is not extracted all thing even want gpt given answer

ChatGPT said:

✓ Here is the updated PDF with the complete, detailed session content including all GPT-generated answers:

 [Download Full_DI_Lifetimes_Session_Clean.pdf](#)

Let me know if you want to include code samples as appendices or convert it into a presentation format!

You said:

give the example of scoped

ChatGPT said:

Certainly! Here's a **clear example of a Scoped service** in a real-world ASP.NET Core application.

Example: UserService with Scoped Lifetime

Use Case:

You want to create a service that manages the **currently logged-in user's** data during the **lifetime of an HTTP request**. You don't want this state shared across different users or requests.

Step 1: Define the Scoped Service Interface and Class

```
csharp
CopyEdit
public interface IUserService
{
    string GetUserId();
    void SetUserId(string userId);
}

public class UserService : IUserService
{
    private string _userId;

    public string GetUserId()
    {
        return _userId;
    }
}
```



```

    public void SetUserId(string userId)
    {
        _userId = userId;
    }
}

```

Step 2: Register it as Scoped in Program.cs

```

csharp
CopyEdit
builder.Services.AddScoped<IUserService, UserService>();

```

Step 3: Inject into a Controller

```

csharp
CopyEdit
[ApiController]
[Route("[controller]")]
public class UserController : ControllerBase
{
    private readonly IUserService _userService;

    public UserController(IUserService userService)
    {
        _userService = userService;
    }

    [HttpGet("{id}")]
    public IActionResult SetUser(string id)
    {
        _userService.SetUserId(id);
        return Ok($"User ID set to: {id}");
    }
}

```

```

[HttpGet("current")]
public IActionResult GetUser()
{
    return Ok($"Current User ID: {_userService.GetUserId()}");
}
}

```

How Scoped Works Here

- For each **HTTP request**, a **new instance** of UserService is created.
- If you call /user/123 then /user/current in the **same request pipeline**, you'll get the same instance and it will remember the ID.
- But across **different requests**, it will **not retain** the ID — that's why it is safe and isolated.

Testing in Swagger/Postman:

1. GET /user/123 → Sets user ID to 123
2. GET /user/current (in the same request) → Returns Current User ID: 123
3. Make another request → Current User ID: will be null (new instance)

Summary of Scoped

Property	Scoped Lifetime
Lifetime	One instance per HTTP request
Shared in Request	 Yes
Shared across Requests	 No
Good For	DbContext, Session/User data

Let me know if you want this implemented with DbContext or middleware for deeper real-world usage!

You said:

GET /user/123 → Sets user ID to 123 GET /user/current (in the same request) → Returns Current User ID: 123 Make another request → Current User ID: will be null (new instance) How you can do this

ChatGPT said:

Excellent question!

You're pointing out something subtle but important:

In ASP.NET Core, each HTTP request is handled separately. So if you make:

- GET /user/123
- then GET /user/current

as **two separate HTTP requests**, even though they happen close together, they will **not share the same scoped instance**. This is the expected behavior.

So How Can We Simulate "Same Request" for Testing Scoped?

There are **two main ways** to demonstrate a scoped instance in a way that shows both SetUserId and GetUserId in the **same request context**:

Option 1: Combine both calls into a single action

This keeps both method calls inside the **same HTTP request**, showing that the scoped service retains state.

csharp
CopyEdit

```
[HttpGet("set-and-get/{id}")]
public IActionResult SetAndGetUser(string id)
{
    _userService.SetUserId(id);
    string currentId = _userService.GetUserId();
    return Ok($"Set and fetched user ID in same request:
{currentId}");
}
```

👉 This will output:

```
sql
CopyEdit
Set and fetched user ID in same request: 123
```

✓ Because both calls happen **inside the same request**, the scoped service behaves as expected.

☑ Option 2: Use Scoped service with a downstream dependency (e.g., a DbContext)

Another way to demonstrate Scoped behavior is through **Entity Framework Core**:

```
csharp
CopyEdit
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));
```

Then inject AppDbContext into services. Even across multiple methods, it shares the same instance within the request.

Why GET /user/123 then GET /user/current doesn't share data?

Because:

- HTTP is **stateless**.
- ASP.NET Core creates a **new request scope** each time.
- Even though UserService is scoped, it's created **per request**.

To persist across multiple requests (i.e. across GET /user/123 and GET /user/current), you'd need to store the data:

- In Session
- In a Cookie
- In a Database
- Or in-memory cache like IMemoryCache (with Singleton)

TL;DR

Scenario	Shared Scoped?	Explanation
Multiple actions in 1 request	✓ Yes	Same DI scope
Multiple HTTP requests	✗ No	Each request gets a new DI scope
Want to persist across requests	✗ (Use session/cache/db instead)	

Want Code That Uses Session or Cache Instead?

Let me know if you'd like an example where the user ID is persisted **across requests** using ISession or IMemoryCache.

You said:

If i used transient here what will happen

ChatGPT said:

Great follow-up! Let's explore **what happens if you use Transient** instead of Scoped in the same example.

Recap: What Does Transient Mean?

A service registered as **Transient** means:

Every time the service is requested (injected), **a new instance** is created — even **within the same HTTP request**.

```
csharp
CopyEdit
builder.Services.AddTransient<IUserService, UserService>();
```

What Happens in Your Example?

You had this controller:

```
csharp
CopyEdit
public class UserController : ControllerBase
{
    private readonly IUserService _userService;

    public UserController(IUserService userService)
    {
        _userService = userService;
    }

    [HttpGet("{id}")]
    public IActionResult SetUser(string id)
    {
        _userService.SetUserId(id);
    }
}
```

```

        return Ok($"User ID set to: {id}");
    }

    [HttpGet("current")]
    public IActionResult GetUser()
    {
        return Ok($"Current User ID: {_userService.GetUserId()}");
    }
}

```

If IUserService is Transient:

1. Each HTTP request = new controller instance.
2. _userService injected into the controller = a fresh instance.
3. SetUserId() is called on one transient instance.
4. Then, GetUserId() is called on another (fresh) transient instance → **state is lost**.

! Even in the same HTTP request, if you inject IUserService into two different classes, both get different instances.

Safer Test Case to See Transient Behavior:

```

csharp
CopyEdit
[HttpGet("test/{id}")]
public IActionResult Test(string id, [FromServices] IUserService
user2)
{
    _userService.SetUserId(id);
    string id1 = _userService.GetUserId(); // from constructor
    string id2 = user2.GetUserId();         // new injection

    return Ok(new
    {
        IdFromConstructor = id1,

```

```

        IdFromActionParam = id2
    });
}

```

If IUserService is **Transient**, the output will be:

```

json
CopyEdit
{
  "IdFromConstructor": "123",
  "IdFromActionParam": null
}

```

Because:

- `_userService` is one instance
- `user2` is another new instance (transient!)
- `SetUserId()` only affected the first one

Summary: What Changes When Using Transient?

Behavior	Scoped	Transient
New per request?	✓ Yes	✓ Yes
Shared within request?	✓ Yes (one instance)	✗ No (always new)
Good for holding request state?	✓ Yes	✗ No (not retained)
Performance	● Moderate	● Can be expensive if overused

When Should You Use Transient?

Use Transient when:

- The service is **stateless**
- Each use should get a **fresh, clean instance**
- Example: formatters, short-lived logic, converters, lightweight utilities

Let me know if you'd like to test it live using a full minimal API sample.

You said:

Safer Test Case to See Transient Behavior: How you will call this

ChatGPT said:

Great! Let's walk through **how to call the "safer test case"** that demonstrates the behavior of a Transient service in ASP.NET Core — especially how it returns different results within the **same HTTP request**.

The Code Recap

```
csharp
CopyEdit
[HttpGet("test/{id}")]
public IActionResult Test(string id, [FromServices] IUserService
user2)
{
    _userService.SetUserId(id);

    string id1 = _userService.GetUserId(); // Injected via constructor
    string id2 = user2.GetUserId();        // Injected via method
parameter

    return Ok(new
    {
        IdFromConstructor = id1,
        IdFromActionParam = id2
    });
}
```

```
}
```

How to Register Transient Service

In `Program.cs` or `Startup.cs`, register `UserService` as `Transient`:

```
csharp
CopyEdit
builder.Services.AddTransient<IUserService, UserService>();
```

How to Call This Endpoint

You can use **Swagger**, **Postman**, or **browser** to call it like this:

Example URL:

```
bash
CopyEdit
GET https://localhost:5001/user/test/123
```

Replace `5001` with your actual ASP.NET Core port.

What Happens:

- Controller constructor receives a **Transient** instance (`_userService`)
- Method parameter `[FromServices] IUserService user2` receives a **new Transient** instance
- The two instances **do not share state**

Expected Output:

```
json
CopyEdit
```

```
{  
  "IdFromConstructor": "123",  
  "IdFromActionParam": null  
}
```

Why?

- You set the value in one instance (_userService)
- You tried to read from another (user2), which was newly created — and **had no idea** about the set operation.

Optional Enhancement: Add GUIDs to Make Instances Obvious

Modify the service to include a GUID to prove they are different:

```
csharp  
CopyEdit  
public class UserService : IUserService  
{  
    private string _userId;  
    private Guid _instanceId = Guid.NewGuid();  
  
    public void SetUserId(string userId) => _userId = userId;  
    public string GetUserId() => _userId;  
    public Guid GetInstanceId() => _instanceId;  
}
```

Update the controller:

```
csharp  
CopyEdit  
[HttpGet("test/{id}")]  
public IActionResult Test(string id, [FromServices] IUserService  
user2)
```

```
{
    _userService.SetUserId(id);

    return Ok(new
    {
        IdFromConstructor = _userService.GetUserId(),
        IdFromActionParam = user2.GetUserId(),
        InstanceFromConstructor = (_userService as
UserService)?.GetInstanceId(),
        InstanceFromParam = (user2 as UserService)?.GetInstanceId()
    });
}
```

Now you'll **see two different GUIDs** — proving two separate instances.